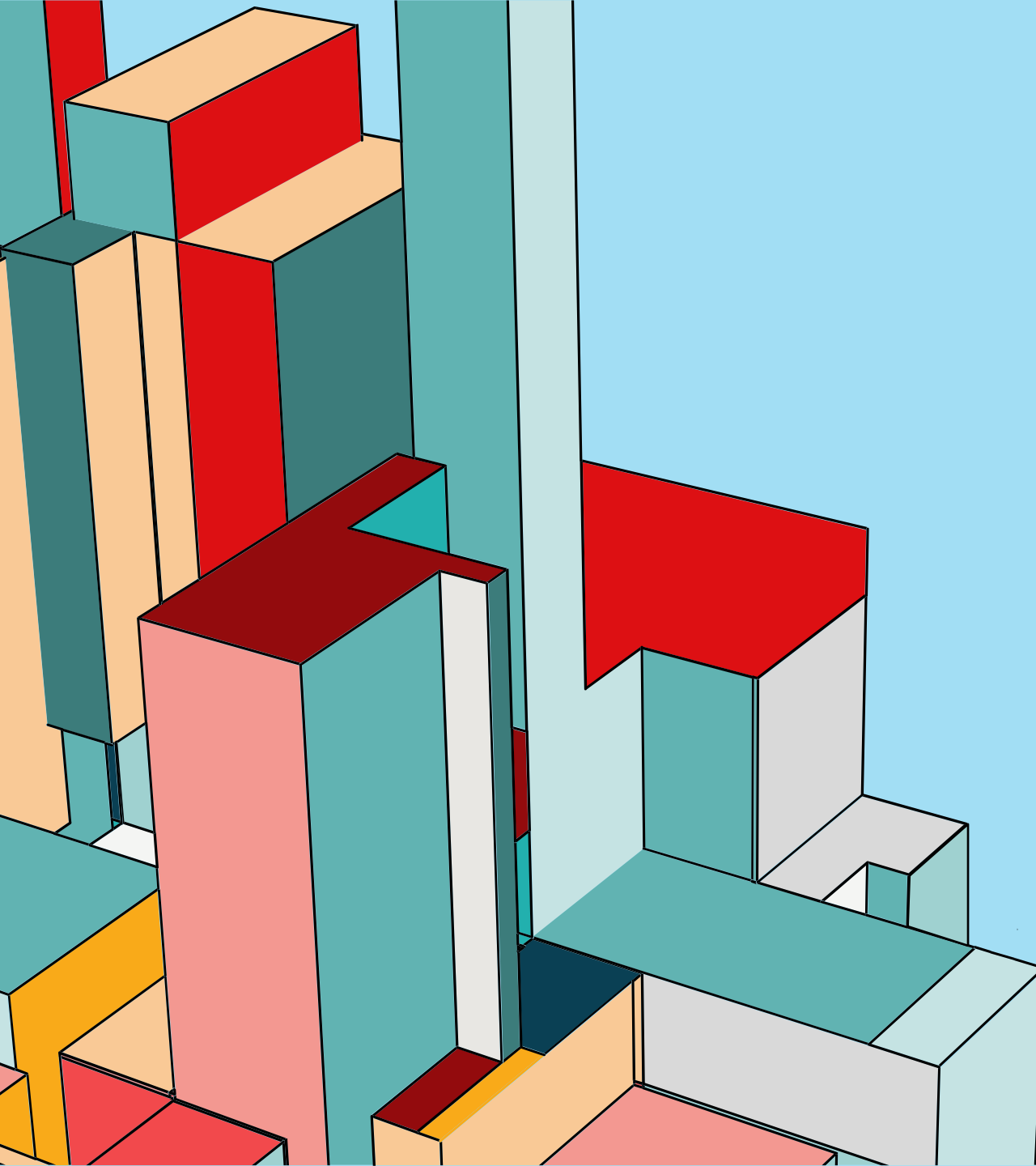




LECTURE 18 **DYNAMIC PROGRAMMING III**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

STRING ALIGNMENT AND EDIT DISTANCE

We define an *edit* as one of three operations: an insertion of a single character, a deletion of a single character, or a substitution of one character for another.

These notions have been around for a long time – they were likely intuitively known to typesetters for centuries – but the rise of DNA/RNA sequencing and other bioinformatics drove a lot of study in this area.

We can then ask how many edits are needed to turn one word into another.

For example: SNOWY and SUNNY. We might use five edits as follows:

–	S	N	O	W	–	Y
S	U	N	–	–	N	Y

STRING ALIGNMENT AND EDIT DISTANCE

The *edit distance* (or *Levenshtein distance*) between two strings is the minimum number of insertions, deletions, and substitutions needed to turn one string into the other.

Technically, there are many different measures that are edit distances (e.g. Hamming distance) – they rely on different types of edits being allowed or not. But in common usage, when someone says “edit distance” they mean this particular metric.

Note that this meets the mathematical definition of a *distance metric*: It’s 0 only when the inputs are identical and positive otherwise, it’s symmetric, and it obeys the Triangle Inequality.

How might we calculate edit distance?

EDIT DISTANCE VIA DYNAMIC PROGRAMMING

Do now: Characterize the recursive substructure of the edit distance problem. Define what the subproblems are and then write the recurrence relation.

Some hints to get you moving:

- As ever, we should first look at reducing the problem size by 1. In this case, consider the last character of BOTH strings.
- You'll need a 2-D array.

EDIT DISTANCE VIA DYNAMIC PROGRAMMING

The recursive structure of the problem can be characterized this way: Consider two strings s and t with lengths i and j respectively. If we examine characters s_i and t_j , what are the possibilities?

1. We align s_i with a spacer. (deletion operation)
2. We align t_j with a spacer. (insertion operation)
3. We align s_i and t_j with each other, possibly needing a substitution.

-	S	N	O	W	-	Y
S	U	N	-	-	N	Y

- Type 1: columns 4 & 5; type 2: columns 1 & 6; type 3: columns 2,3,7

EDIT DISTANCE VIA DYNAMIC PROGRAMMING

1. We align s_i with a spacer. (deletion operation)
2. We align t_j with a spacer. (insertion operation)
3. We align s_i and t_j with each other, possibly needing a substitution.

This suggests the following recurrence relation:

$$OPT(i, j) = \min \begin{cases} OPT(i-1, j) + 1 \\ OPT(i, j-1) + 1 \\ OPT(i-1, j-1) + \begin{cases} 0 & \text{if } s_i == t_j \\ 1 & \text{if } s_i \neq t_j \end{cases} \end{cases}$$

Our base cases will be $OPT(0,0) = 0$; $OPT(i,0) = i$; $OPT(0,j) = j$

EDIT DISTANCE VIA DYNAMIC PROGRAMMING

$$OPT(i, j) = \min \text{ of } \begin{cases} OPT(i-1, j) + 1 \\ OPT(i, j-1) + 1 \\ OPT(i-1, j-1) + \begin{cases} 0 & \text{if } s_i == t_j \\ 1 & \text{if } s_i \neq t_j \end{cases} \end{cases}$$

Our base case will be $OPT(0,0) = 0$; $OPT(i, 0) = i$; $OPT(0, j) = j$

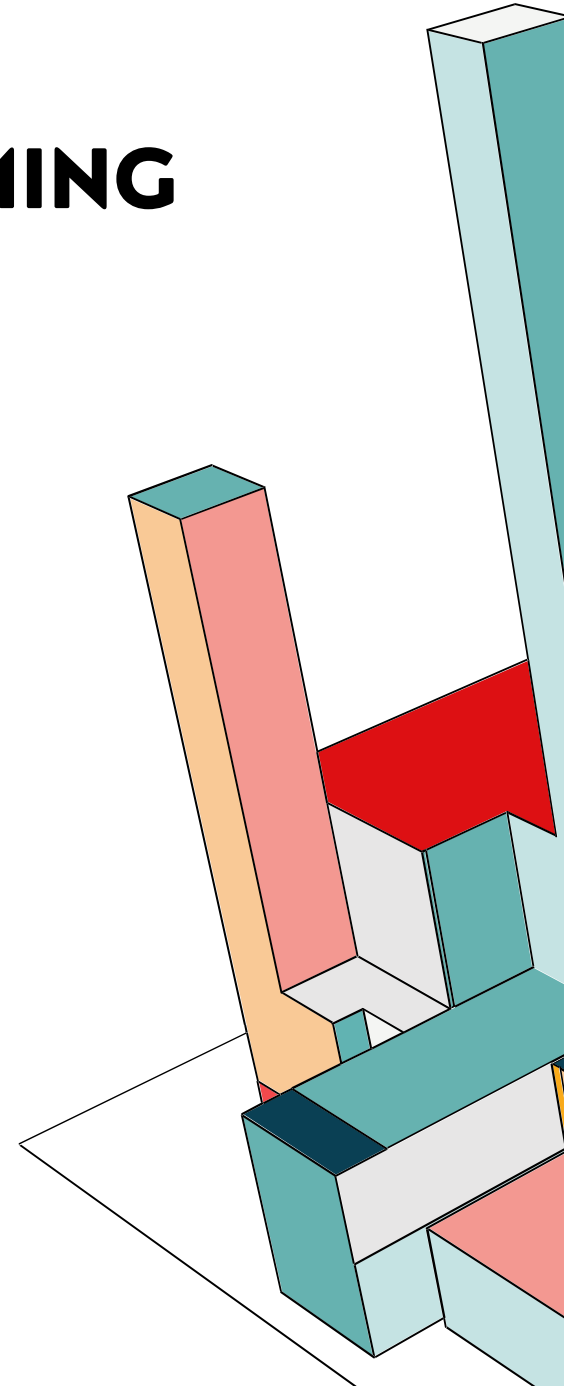
Do now: Given the above recurrence relation, write the pseudocode for this algorithm.

EDIT DISTANCE VIA DYNAMIC PROGRAMMING

```
# For convenience, string indices start at 1
def edIT(s,t): # len(s) = m; len(t) = n
    Declare OPT as an (m+1) by (n+1) array
    Initialize OPT[0][i] = i and OPT[j][0] = j
    for i = 1 to m:
        for j = 1 to n:
            a = 1 + OPT[i-1][j]
            b = 1 + OPT[i][j-1]
            c = OPT[i-1][j-1]
            if s[i] != t[j]:
                c = c + 1
            OPT[i][j] = min(a,b,c)
    return OPT[m][n]
```



Once again,
order of loops
doesn't matter
in this problem.



EDIT DISTANCE EXAMPLE

Do now: Fill in the edit distance DP array for the inputs CERTAIN and RECURSION.

	--	R	E	C	U	R	S	I	O	N
--	0	1	2	3	4	5	6	7	8	9
C	1									
E	2									
R	3									
T	4									
A	5									
I	6									
N	7									

SUBMITTY POLL #1

What is the edit distance
between CERTAIN and
RECURSION?



EDIT DISTANCE EXAMPLE

Do now: Fill in the edit distance DP array for the inputs CERTAIN and RECURSION.

	--	R	E	C	U	R	S	I	O	N
--	0	1	2	3	4	5	6	7	8	9
C	1	1	2	2	3	4	5	6	7	8
E	2	2	1	2	3	4	5	6	7	8
R	3	2	2	2	3	3	4	5	6	7
T	4	3	3	3	3	4	4	5	6	7
A	5	4	4	4	4	4	5	5	6	7
I	6	5	5	5	5	5	5	5	6	7
N	7	6	6	6	6	6	6	6	6	6

RECONSTRUCTING THE SOLUTION

From the goal entry, work backwards (up and left), taking the smallest entry each time. Since there are multiple six-edit approaches, there are multiple paths through the array.

--CERTAIN
RECURSION

CERTA-I-N
RECURSION

	--	R	E	C	U	R	S	I	O	N
--	0	1	2	3	4	5	6	7	8	9
C	1	1	2	2	3	4	5	6	7	8
E	2	2	1	2	3	4	5	6	7	8
R	3	2	2	2	3	3	4	5	6	7
T	4	3	3	3	3	4	4	5	6	7
A	5	4	4	4	4	4	5	5	6	7
I	6	5	5	5	5	5	5	5	6	7
N	7	6	6	6	6	6	6	6	6	6

MULTIPLYING MATRICES

Our next problem involves matrix multiplication.

Quick refresher:

- Can only multiply matrices $A \times B$ when the number of columns of A equals the number of rows of B .
 - If A is $r \times s$ and B is $s \times t$, then the product will be $r \times t$.
- Each entry of the result involves multiplying a row of A pairwise by a column of B and then summing the products.

$$\begin{bmatrix} 4 & 9 \\ 2 & 5 \\ 7 & 1 \end{bmatrix} \times \begin{bmatrix} 3 & 10 & 2 \\ 5 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 4 \cdot 3 + 9 \cdot 5 & 4 \cdot 10 + 9 \cdot 6 & 4 \cdot 2 + 9 \cdot 8 \\ 2 \cdot 3 + 5 \cdot 5 & 2 \cdot 10 + 5 \cdot 6 & 2 \cdot 2 + 5 \cdot 8 \\ 7 \cdot 3 + 1 \cdot 5 & 7 \cdot 10 + 1 \cdot 6 & 7 \cdot 2 + 1 \cdot 8 \end{bmatrix}$$

MULTIPLYING MATRICES

Matrix multiplication is not commutative: $A \times B \neq B \times A$

But it is associative: $A \times (B \times C) = (A \times B) \times C$

Quick check-in: Let A be $r \times s$, B be $s \times t$, and C be $t \times u$. Verify for yourself that either order of multiplication produces an $r \times u$ matrix.

For longer chains, the number of possible orderings increases exponentially. For example, $A \times B \times C \times D \times E$ can be parenthesized in 14 different ways:

$A \times (B \times (C \times (D \times E)))$, $A \times (B \times ((C \times D) \times E))$, $A \times ((B \times C) \times (D \times E))$,
 $A \times ((B \times (C \times D)) \times E)$, $A \times (((B \times C) \times D) \times E)$, $(A \times B) \times (C \times (D \times E))$,
 $(A \times B) \times ((C \times D) \times E)$, $((A \times B) \times C) \times (D \times E)$, $(A \times (B \times C)) \times (D \times E)$, ...

MULTIPLYING MATRICES

While all of these parenthesizations lead to the same outcome, some of them will take a lot longer than others.

The critical operation in matrix multiplication is the pairwise multiplication of the individual elements; in general, multiplying an $r \times s$ matrix by an $s \times t$ matrix requires rst scalar products.*

* We saw that divide-and-conquer approaches like Strassen's algorithm can improve upon this, but the point still holds.

MULTIPLYING MATRICES – ORDER MATTERS! (FOR EFFICIENCY, ANYWAY)

Let A be 50×20 , B be 20×1 , C be 1×10 , and D be 10×100 .

Then the number of scalar multiplications required for various parenthesizations is as follows:

$$A \times ((B \times C) \times D): 20 \cdot 1 \cdot 10 + 20 \cdot 10 \cdot 100 + 50 \cdot 20 \cdot 100 = 120,200$$

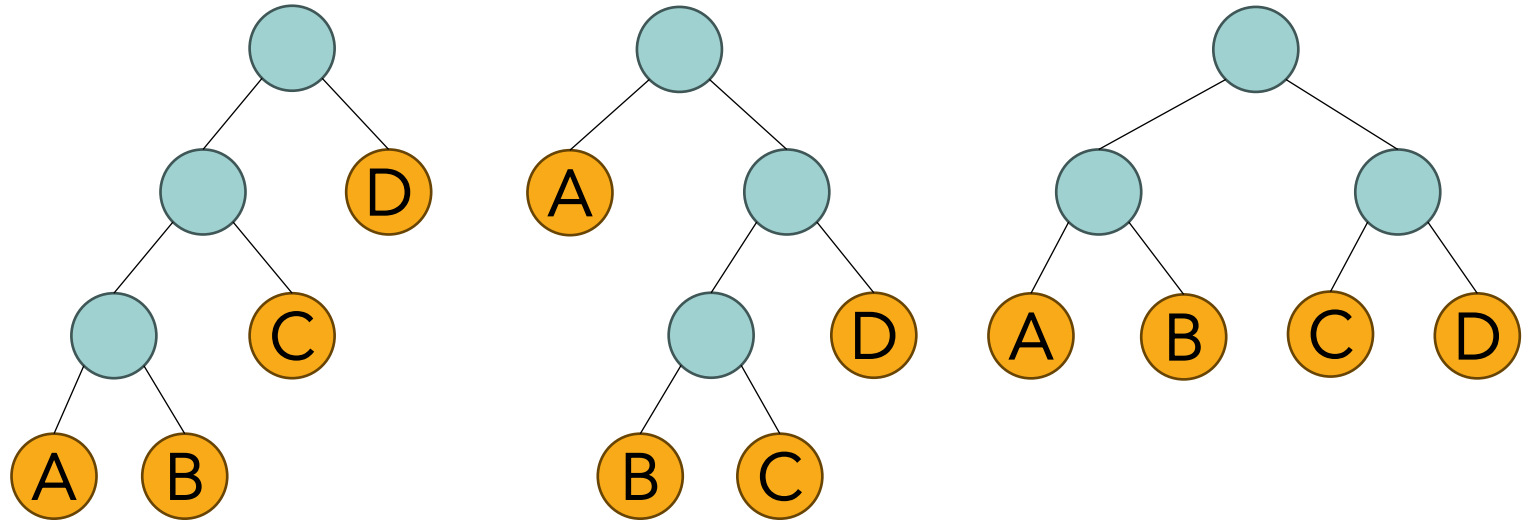
$$(A \times (B \times C)) \times D: 20 \cdot 1 \cdot 10 + 50 \cdot 20 \cdot 10 + 50 \cdot 10 \cdot 100 = 60,200$$

$$(A \times B) \times (C \times D): 50 \cdot 20 \cdot 1 + 1 \cdot 10 \cdot 100 + 50 \cdot 1 \cdot 100 = 7,000$$

The general task, then, is to find the best parenthesization given a matrix multiplication problem $A_1 \times A_2 \times \cdots \times A_n$ with dimensions $d_0 \times d_1, d_1 \times d_2, \dots, d_{n-1} \times d_n$.

Question: What data structure best represents a parenthesization?

TREES FTW.

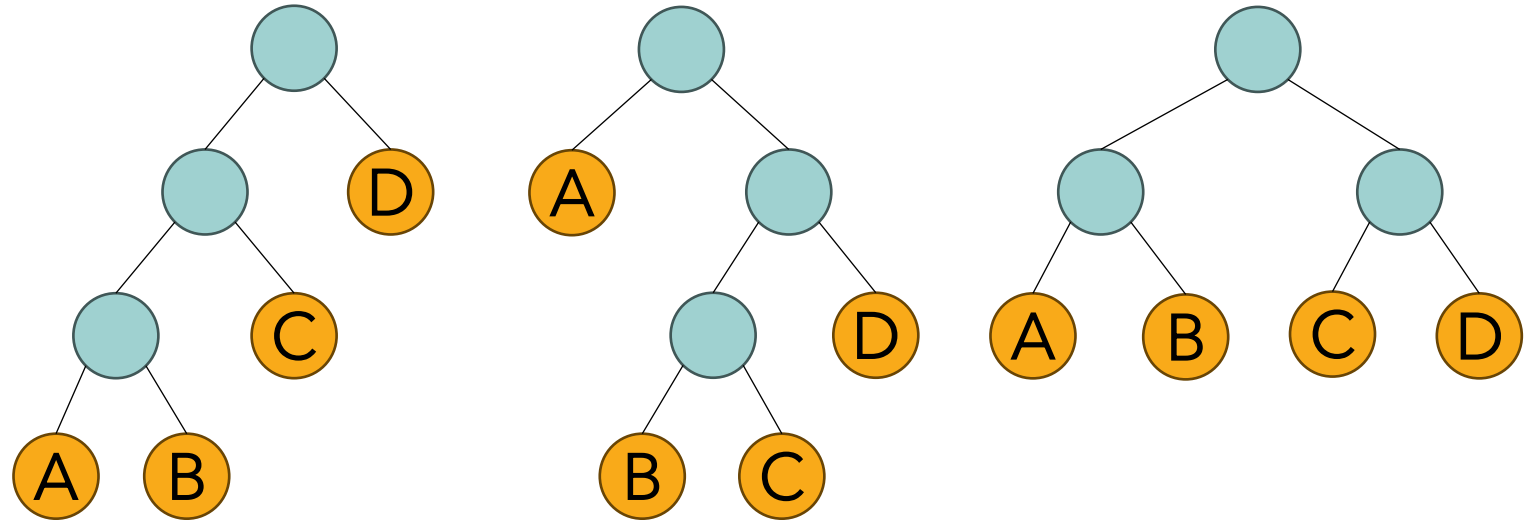


The first tree represents $((A \times B) \times C) \times D$, the second $A \times ((B \times C) \times D)$, and the third $(A \times B) \times (C \times D)$.

In general, we can construct a tree in which the starting matrices are the leaves and each interior node represents one of the intermediate products, with the root node being the final result.

How can we turn this structure into a dynamic programming algorithm?

DYNAMIC PROGRAMMING WITH TREES

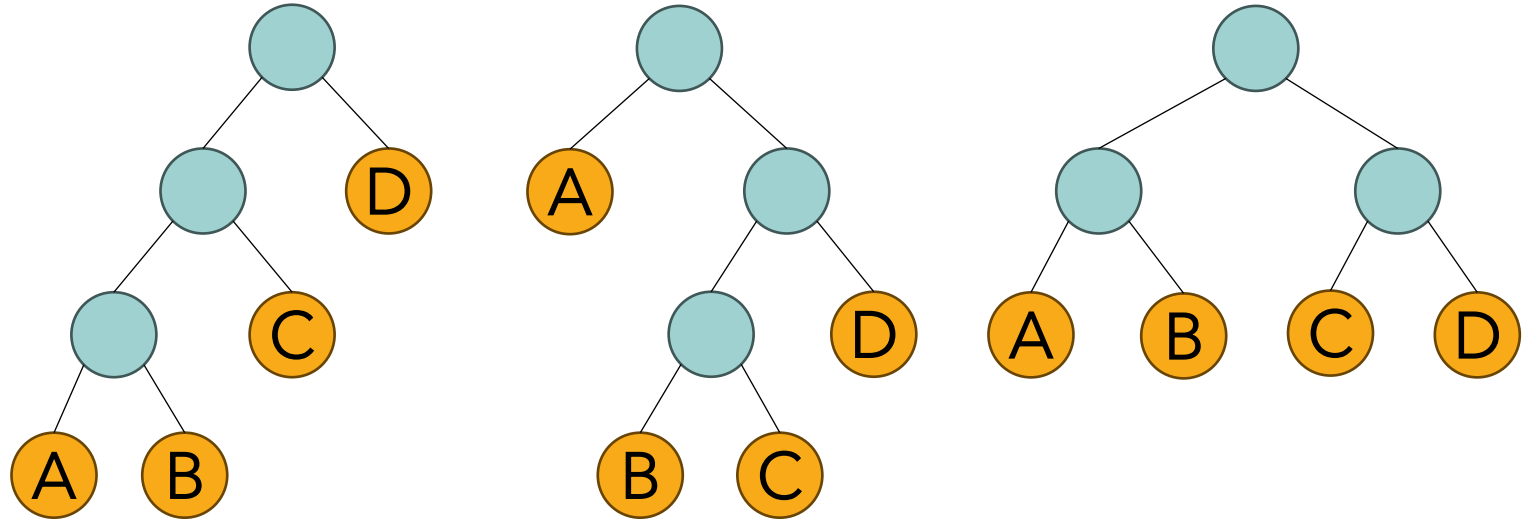


First, observe that we do have optimal substructure: The only way we could possibly be calculating the root node optimally is if we are also calculating each subtree optimally.

If a subtree is not a leaf, then it is an intermediate product.

Since matrix multiplication is not commutative, these intermediate products must involve matrices that appear consecutively in our original problem.

DYNAMIC PROGRAMMING WITH TREES

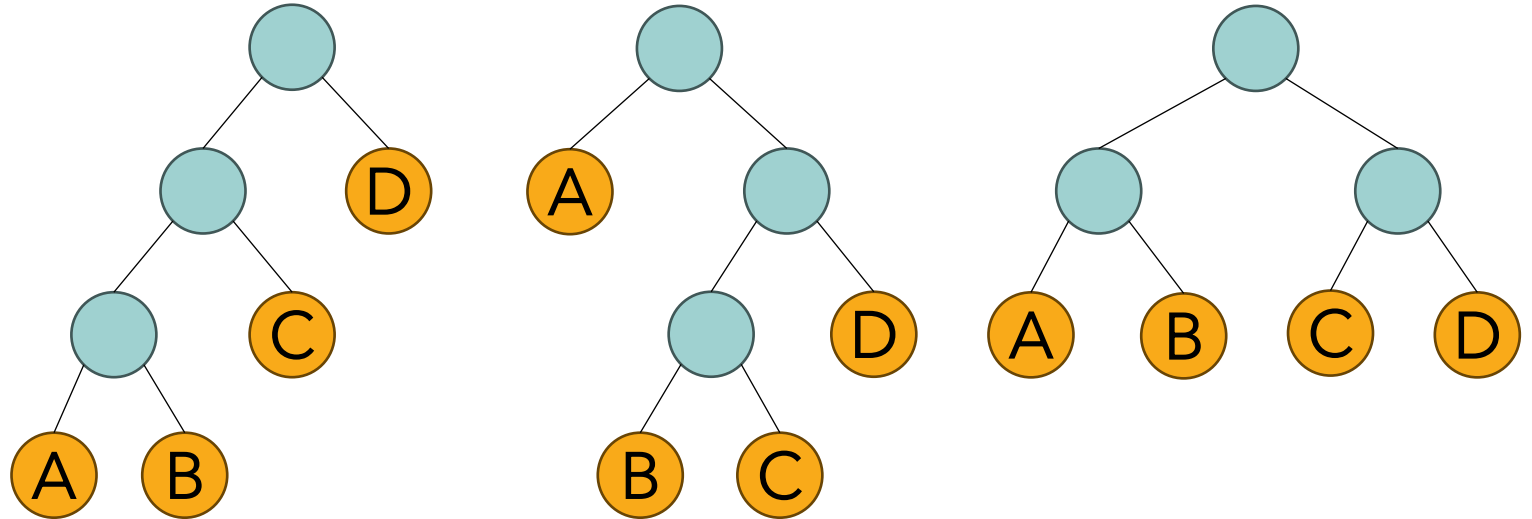


An optimal subtree is a product $A_i \times A_{i+1} \times \cdots \times A_j$ for some indices i and j . Thus we can define $OPT(i, j)$ as the minimum cost of multiplying $A_i \times \cdots \times A_j$.

Besides optimal subtrees, we also need to make the optimal split into subtrees – this is the core decision at each step. Once again, we'll simply try everything. Let k be the split point:

$$OPT(i, j) = \min_k OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j$$

DYNAMIC PROGRAMMING WITH TREES



$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j$$

The last term here is the cost of the final matrix multiplication (i.e. the root of this subtree) if we split at index k .

$OPT(i, k)$ will multiply matrices $A_i \times \cdots \times A_k$, resulting in a $d_{i-1} \times d_k$ matrix. Similarly, $OPT(k + 1, j)$ results in a $d_k \times d_j$ matrix. Multiplying these two matrices together requires $d_{i-1}d_kd_j$ scalar multiplications.

PSEUDOCODE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

```
def matrixMultAssoc(n,d): # d is the array of dimensions
    Declare OPT and initialize OPT[i][i] = 0
    for s = 1 to n-1:      # s size of subproblem, which is
        for i = 1 to n-s:  # the number of matrix multiplications
            j = i + s
            best = ∞
            for k = i to j-1:
                val = OPT[i][k] + OPT[k+1][j] + d[i-1]*d[k]*d[j]
                if val < best:
                    best = val
            OPT[i][j] = best
    return OPT[1][n]
```

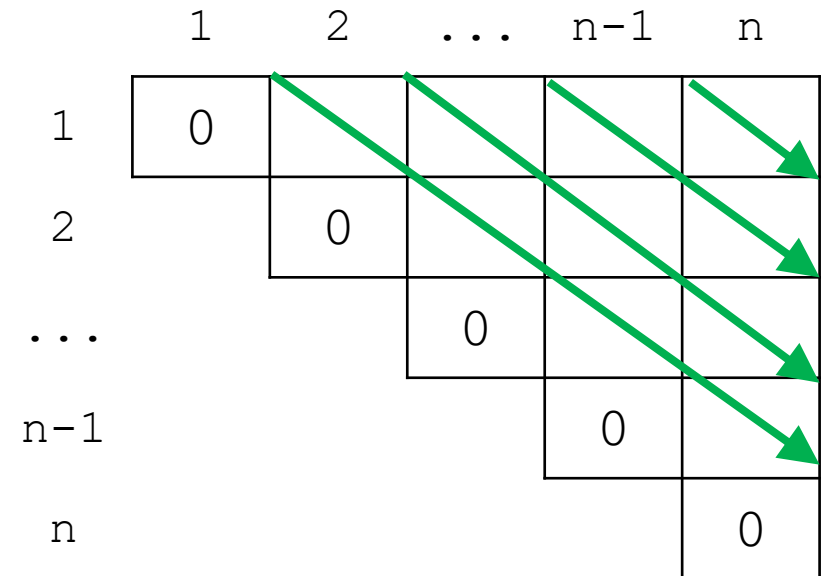
DP TABLE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

Note that since we can never have $j < i$, this gives a slightly different table than we're used to – it's two-dimensional, but we're only going to fill half of it.

The order in which we fill it is also a bit unusual – we sweep along diagonals to the top right corner.

Our runtime is $O(n^3)$: we have a quadratic number of boxes to fill, and each one is a linear-time calculation.



DP TABLE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

Let $d = [5, 10, 50, 2, 20, 4]$.

(That is, A_1 is 5×10 , A_2 is 10×50 , etc.)

The first diagonal is easy – it's just product of each set of three consecutive values in d .

	1	2	3	4	5
1	0				
2		0			
3			0		
4				0	
5					0

DP TABLE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

Let $d = [5, 10, 50, 2, 20, 4]$.

The next diagonal are the size-2 subproblems - we have two multiplications and thus two possible splits to consider.

$$OPT(1, 3) = \min(\textcolor{teal}{OPT(1, 1)} + \textcolor{teal}{OPT(2, 3)} + 5 \cdot 10 \cdot 2, \textcolor{violet}{OPT(1, 2)} + \textcolor{violet}{OPT(3, 3)} + 5 \cdot 50 \cdot 2)$$

	1	2	3	4	5
1	0	2500			
2		0	1000		
3			0	2000	
4				0	160
5					0

DP TABLE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

Let $d = [5, 10, 50, 2, 20, 4]$.

The next diagonal are the size-2 subproblems - we have two multiplications and thus two possible splits to consider.

$$OPT(1, 3) = \min(\textcolor{green}{OPT(1, 1)} + \textcolor{green}{OPT(2, 3)} + 5 \cdot 10 \cdot 2, \textcolor{violet}{OPT(1, 2)} + \textcolor{violet}{OPT(3, 3)} + 5 \cdot 50 \cdot 2)$$

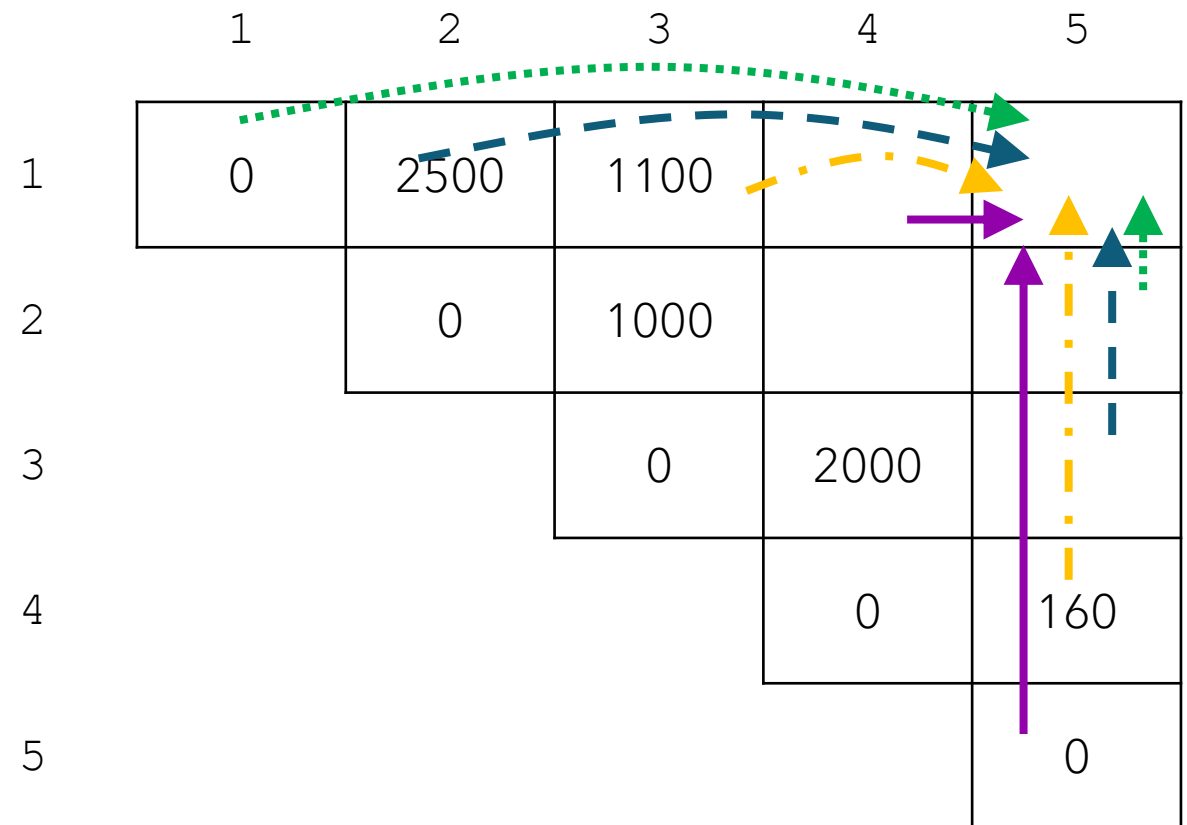
	1	2	3	4	5
1	0	2500	1100		
2		0	1000		
3			0	2000	
4				0	160
5					0

DP TABLE FOR MATRIX CHAINING

$$OPT(i, j) = \min_{k: i \leq k < j} OPT(i, k) + OPT(k + 1, j) + d_{i-1}d_kd_j; \quad OPT(i, i) = 0$$

Let $d = [5, 10, 50, 2, 20, 4]$.

At the last box, we have four different possibilities to look at. (And, in general, we would have n choices in a problem with n matrices.)



BEFORE YOU LEAVE

Please complete this feedback form about today's class.

